ТОО ”Колледж Хекслет”

Лабораторная работа №9

Транзакции и целостность данных

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_3. Транзакции и целостность данных.

Цель: Реализовать создание Order и связанных OrderItem в одной транзакции Prisma, проверить rollback при ошибке и изучить ограничения целостности PostgreSQL.

Условие: RO4_3/lesson9/ЛР-9.pdf

Требования задания

• POST /api/orders создаёт Order и все OrderItem.

• Все операции выполняются внутри prisma.$transaction().

• Внутри транзакции используется объект tx.

• Service проверяет существование User и Product.

• Service проверяет inStock.

• Цена OrderItem берётся из Product.

• Успешный запрос фиксируется полностью.

• Ошибка любой позиции приводит к полному rollback.

• В базе не остаются частично созданные данные.

• Студент может объяснить роль PK, FK, UNIQUE, NOT NULL и CHECK.

Ход выполнения

Создание заказа и позиций выполняется в одном Prisma $transaction callback через tx. Сервер получает цену из Product, проверяет пользователя и наличие товара. Успех фиксирует все записи; ошибка второй позиции откатывает уже созданный Order и первую позицию. Валидация запрещает пустой заказ, нулевое количество и повтор productId; ограничения PostgreSQL защищают данные дополнительно.

Проект: RO4_3/lesson9

Запуск: npm install; npm start. Для PostgreSQL параметры подключения задаются в .env по образцу .env.example. Порядок применения SQL и миграций приведён в README.md проекта.

services/orderService.js

```
services/orderService.js
```

```
import { prisma } from '../config/prisma.js';
import { AppError } from '../errors/AppError.js';
const relations = {
    user: { select: { id: true, name: true, email: true } },
    items: { include: { product: { select: { id: true, name: true } } } }
};
export function findOrderById(id) {
    return prisma.order.findUnique({ where: { id }, include: relations });
}
export async function getOrderDetails(id) {
    const order = await findOrderById(id);
    if (!order) return null;
    return { ...order, total: order.items.reduce((sum, item) => sum + item.price *
item.quantity, 0) };
}
export function findOrdersByProduct(productId) {
    return prisma.order.findMany({ where: { items: { some: { productId } } }, include:
relations, orderBy: { id: 'asc' } });
}
export function getAllOrders() {
    return prisma.order.findMany({ include: relations, orderBy: { id: 'asc' } });
}
export async function createOrder(userId, items) {
    return prisma.$transaction(async tx => {
        const user = await tx.user.findUnique({ where: { id: userId } });
        if (!user) throw new AppError('User not found', 404);
        const order = await tx.order.create({ data: { userId, status: 'new' } });
        for (const item of items) {
            const product = await tx.product.findUnique({ where: { id: item.productId }
});
            if (!product || !product.inStock) throw new AppError(`Product
${item.productId} is unavailable`, 409);
            await tx.orderItem.create({ data: { orderId: order.id, productId:
product.id, quantity: item.quantity, price: product.price } });
        }
```

Листинг 1. services/orderService.js (часть 1)

```
services/orderService.js
```

```
        const result = await tx.order.findUnique({ where: { id: order.id }, include:
relations });
        return { ...result, total: result.items.reduce((sum, item) => sum + item.price *
 item.quantity, 0) };
    });
}
```

Листинг 2. services/orderService.js (часть 2)

middleware/validateOrder.js

middleware/validateOrder.js

```
import { AppError } from '../errors/AppError.js';
export function validateOrder(req, res, next) {
    const { userId, items } = req.body || {};
    const validId = id => Number.isSafeInteger(id) && id > 0;
    if (!validId(userId) || !Array.isArray(items) || !items.length || items.length > 100) return next(new AppError('userId and non-empty items (max 100) required', 400));
    const ids = new Set();
    for (const item of items) {
        if (!item || !validId(item.productId) || !validId(item.quantity) || item.quantity > 2147483647 || ids.has(item.productId)) return next(new AppError('Invalid or duplicate order item', 400));
        ids.add(item.productId);
    }
    next();
}
```

Листинг 3. middleware/validateOrder.js

controllers/orderController.js

controllers/orderController.js

```
import * as service from '../services/orderService.js';
import { AppError } from '../errors/AppError.js';
export async function getOrderById(req, res, next) {
    try {
        const order = await service.getOrderDetails(Number(req.params.id));
        if (!order) return next(new AppError('Order not found', 404));
        res.json(order);
    } catch (error) { next(error); }
}
export async function getOrders(req, res, next) {
    try {
        if (req.query.productId !== undefined && (!Number.isSafeInteger(Number(req.query.productId)) || Number(req.query.productId) <= 0)) throw new AppError('Invalid productId', 400);
        res.json(req.query.productId ? await service.findOrdersByProduct(Number(req.query.productId)) : await service.getAllOrders());
    } catch (error) { next(error); }
}
export async function createOrderController(req, res, next) {
    try { res.status(201).json(await service.createOrder(req.body.userId, req.body.items)); }
    catch (error) { next(error); }
}
```

Листинг 4. controllers/orderController.js

routes/orderRoutes.js

routes/orderRoutes.js

```
import { Router } from 'express';
import { getOrders, getOrderById, createOrderController } from
'../controllers/orderController.js';
import { validateId } from '../middleware/validateId.js';
import { validateOrder } from '../middleware/validateOrder.js';
const router = Router();
router.param('id', validateId);
router.get('/', getOrders);
router.get('/:id', getOrderById);
router.post('/', validateOrder, createOrderController);
export default router;
```

Листинг 5. routes/orderRoutes.js

Проверка результата

Проверено: 05.11.2026 13:38 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Реальный PostgreSQL

• HTTP endpoints

• CRUD сохраняется в PostgreSQL

• UNIQUE и FOREIGN KEY; категории перенесены без потери товаров

• Связи 1:1, 1:N, M:N; CHECK(quantity > 0); составной ключ

• include/select, relation filter some, историческая сумма заказа

• Миграции description/slug применены; повторный seed без дублей

• Атомарный заказ COMMIT/ROLLBACK; цена из БД; валидация позиций

• Prisma Studio открыт в Chromium; таблица products содержит Keyboard

HTTP-запросы

GET /api/products → 200

GET /api/products/1 → 200

GET /api/products/999999 → 404

GET /api/products/abc → 400

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?minPrice=abc → 400

GET /api/products?inStock=maybe → 400

GET /api/products?minPrice=500&maxPrice=100 → 400

POST /api/products → 403

POST /api/products → 400

POST /api/products → 201

PATCH /api/products/6 → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

POST /api/products → 409

GET /api/orders/1 → 200

GET /api/orders?productId=1 → 200

GET /api/orders/999999 → 404

POST /api/products → 201

DELETE /api/products/8 → 200

POST /api/orders → 201

POST /api/orders → 409

POST /api/orders → 400

POST /api/orders → 400

POST /api/orders → 400

POST /api/orders → 404

GET /api/unknown → 404

Примеры ответов

POST /api/products → 201


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 65000,
  "description": null,
  "categoryId": 1,
  "inStock": true
}
```

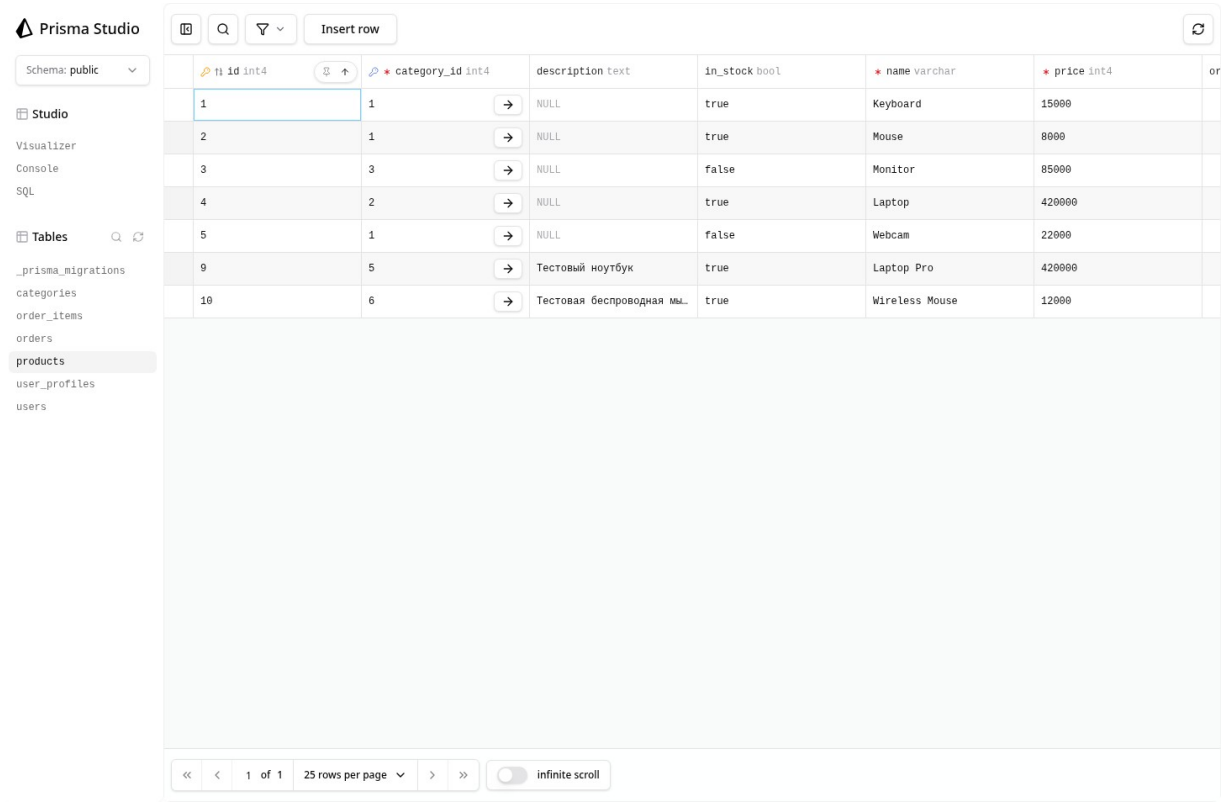

PATCH /api/products/6 → 200


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 42000,
  "description": null,
  "categoryId": 1,
  "inStock": false
}
```


DELETE /api/products/6 → 200


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 42000,
  "description": null,
  "categoryId": 1,
  "inStock": false
}
```

Проверка данных в Prisma Studio



Prisma Studio: таблицы и данные проверенной учебной базы.

Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.